

SAFE SECURITY AUDIT

Rapport statique et dynamique non destructif

Élément	Valeur
Date de l'audit	21 septembre 2026
Branche / commit	main / cbd8417
État Git	Arbre de travail propre ; aucune modification appliquée
Périmètre	API Laravel active, frontend React/TypeScript, Express legacy, configuration Render, dépendances
Environnement dynamique	Local uniquement ; PostgreSQL Render et données de production non interrogés

Conclusion exécutive : audit terminé — aucune modification effectuée. Un défaut d'autorisation SEC-001 est confirmé dans le code de la route de changement de mot de passe entreprise. Les vérifications de production PostgreSQL/Render restent non exécutées.

1. État général

- **Architecture observée** : Laravel + Eloquent est l'API active ; PostgreSQL est la base cible de production ; Express reste legacy et réversible.
- **Local** : l'API a répondu sur le serveur PHP intégré avec une base SQLite locale. Cette différence interdit de conclure à la parité PostgreSQL de production.
- **Render** : la configuration déclarative demande APP_DEBUG=false, DB_CONNECTION=pgsql, DATABASE_URL liée à la base Render, cookie sécurisé et disque persistant pour les fichiers numériques. La configuration effective et les données Render n'ont pas été modifiées ni interrogées.
- **Git** : branche main, commit cbd8417, origin/main alignée, diff vide au moment du contrôle.
- **Règle de sécurité appliquée** : aucun mot de passe, compte, paiement, webhook métier, upload dangereux, secret, donnée, migration ou déploiement n'a été modifié.

Tests et contrôles exécutés

Contrôle	Résultat	Portée
Dependency audit	0 critique, 1 élevé, 3 modérés, 2 faibles	Présence de risques dans le graphe de dépendances ; aucune mise à jour
SAST	Analyse incomplète, 0 résultat remonté	Ne constitue pas une preuve d'absence de vulnérabilité
HoundDog	0 vulnérabilité	Analyse statique des secrets/données
Tests déjà validés avant cette phase	234 tests Laravel / 1 720 assertions ; 147 tests frontend ; typecheck, build Vite et syntaxe PHP	Résultats antérieurs conservés ; non relancés pendant l'audit
Tests HTTP locaux	GET, HEAD et OPTIONS sûrs ; webhook sans signature rejeté	Aucune mutation métier

2. Résumé

Les alertes sont séparées entre défauts applicatifs et présence de paquets signalés. Une alerte de dépendance confirme la présence dans le graphe, mais pas l'exploitation dans le chemin runtime Laravel sans analyse de reachability complémentaire.

Indicateur	Compte
Signalements structurés	10 : 4 observations applicatives + 6 alertes de dépendances
CRITIQUES	0
ÉLEVÉS	2 : SEC-001 + 1 alerte js-yaml
MOYENS	6 : SEC-002, SEC-003, CORS + 3 alertes modérées
FAIBLES	2 alertes de dépendances
CONFIRMÉS	2 : SEC-001 ; présence des alertes Dependency audit
POTENTIELS	3 : SEC-002, SEC-003, CORS
TEST NON EXÉCUTÉ	Production Render/PostgreSQL, mutation de mot de passe, paiements, upload et rejouement webhook

Attention : les comptes de sévérité incluent les six alertes de dépendances individuelles, tandis que les contrôles applicatifs sont regroupés par sujet.

3. Vulnérabilités et risques

SEC-001 — Changement du mot de passe company/admin insuffisamment restreint

Sévérité : ÉLEVÉE

Statut : CONFIRMÉ

Zone : PATCH /api/auth/company-password — AuthController::updateCompanyPassword

Preuve : La route applique uniquement maximus.auth. Pour tout acteur non maximus_admin, le contrôleur prend automatiquement actor.companyId, recherche le premier compte actif company_admin de cette entreprise, remplace son hash et invalide ses sessions. Aucun contrôle explicite du rôle company_admin ni Policy dédiée n'est appliqué avant cette opération.

Reproduction sûre : Ne pas envoyer de requête de changement et ne pas changer de mot de passe réel. La preuve statique suffit : un acteur authentifié avec un rôle employee peut appeler la route sans companyId ; le chemin serveur atteint la mise à jour de l'administrateur de son entreprise.

Impact potentiel : Un compte employé compromis pourrait provoquer une prise de contrôle ou un déni d'accès du compte administrateur entreprise. La réauthentification n'est pas demandée par cette route.

Correction proposée : Ajouter une autorisation serveur explicite : maximus_admin ou company_admin autorisé, avec vérification de périmètre ; exiger une réauthentification récente pour le compte sensible ; ajouter un test négatif employee/sector_manager qui vérifie 403 sans modifier de données.

Risque de correction : Risque de bloquer un parcours d'administration actuellement utilisé par une interface existante ; cartographier les rôles attendus avant correction et tester sur une base PostgreSQL dédiée.

SEC-002 — Profil d'installation public trop détaillé

Sévérité : MOYENNE

Statut : POTENTIEL

Zone : GET /api/installation — InstallationContext::publicProfile

Preuve : La réponse publique contient mode, companyOnly, installationId, canonicalUrl/loginUrl, états ready/adminLoginEnabled/registrationEnabled et, en mode isolé, une synthèse de synchronisation. En mode central local, aucun secret ni identifiant d'entreprise n'a été observé, mais le contrat autorise ces champs en mode dédié.

Reproduction sûre : GET local sans authentification : réponse 200 et profil JSON ; aucune écriture. La production n'a pas été interrogée.

Impact potentiel : Reconnaissance de l'installation, de l'entreprise et de son état opérationnel. Aucun secret n'a été observé dans la réponse locale ; l'exposition est donc classée potentielle, pas comme fuite confirmée.

Correction proposée : Limiter le profil aux informations indispensables au bootstrap public ; retirer de la réponse publique installationId, erreurs de synchronisation détaillées et identifiants d'entreprise si le frontend n'en a pas besoin.

Risque de correction : Casser l'amorçage de l'interface ou les parcours de connexion personnalisés si un champ est consommé implicitement ; vérifier les clients avant réduction.

SEC-003 — Endpoint de santé public révélant l'état interne

Sévérité : MOYENNE

Statut : POTENTIEL

Zone : GET /api/healthz

Preuve : La réponse 200 locale contient ok=true, database=true, status=OPERATIONAL, applicationVersion=unknown, syncProtocolVersion=2 et checkedAt. Le endpoint est aussi le health check Render et ne renvoie pas de trace ni de chaîne de connexion.

Reproduction sûre : GET et HEAD locaux sans authentification : 200. OPTIONS annonce GET,HEAD. Aucun accès production.

Impact potentiel : Divulcation limitée d'état et de versions utile à la reconnaissance. L'usage pour le health check est légitime ; la gravité reste potentielle.

Correction proposée : Conserver le statut attendu par Render mais réduire la réponse publique à un état minimal ; réserver les détails de version/protocole au diagnostic authentifié.

Risque de correction : Render ou un moniteur externe peut dépendre de la forme JSON actuelle ; valider le contrat avant changement.

SEC-004 — CORS wildcard à revalider

Sévérité : MOYENNE

Statut : POTENTIEL

Zone : Réponses API locales GET/HEAD/OPTIONS

Preuve : Les réponses observées portent Access-Control-Allow-Origin: *. Le frontend utilise credentials: include ; avec un wildcard, les navigateurs refusent normalement la lecture credentialed, ce qui limite le scénario immédiat, mais la politique reste trop large pour une API authentifiée par cookie.

Reproduction sûre : GET/HEAD/OPTIONS locaux depuis curl, sans cookie et sans mutation. Aucun scénario cross-origin credentialed n'a été lancé.

Impact potentiel : Accès cross-origin public élargi et risque de régression si une future configuration autorise les credentials ou un autre mode d'authentification.

Correction proposée : Définir explicitement les origines ERP/boutique nécessaires, désactiver le wildcard pour les routes privées et documenter le comportement des routes publiques.

Risque de correction : Bloquer des boutiques ou installations légitimes si la liste d'origines est incomplète ; établir la matrice des domaines avant modification.

Alerte de dépendances — DEP-001

Sévérité agrégée : ÉLEVÉE / MOYENNE / FAIBLE selon le paquet. **Statut** : CONFIRMÉ pour la présence dans le graphe ; exploitabilité runtime non confirmée.

Le dependency audit a trouvé 1 alerte élevée, 3 modérées et 2 faibles. Le paquet explicitement identifié comme élevé est **js-yaml 4.3.1**. Les autres paquets explicitement signalés sont **esbuild 0.18.20/0.27.3**, **nanoid 3.3.16** et **qs 6.15.3**. Deux alertes faibles supplémentaires sont conservées dans le compte du scanner sans inventer leur nom à partir de la synthèse disponible.

- **Preuve** : résultat du scanner de dépendances ; aucune dépendance n'a été mise à jour.
- **Impact** : dépend de l'utilisation effective des chemins vulnérables, principalement outil de build ou dépendance transitive ; aucune exploitation n'a été démontrée.
- **Correction proposée** : examiner les chemins directs/transitifs, appliquer les versions corrigées dans une branche dédiée après validation, puis relancer audit, tests, typecheck et build.
- **Risque** : changement de résolution npm, incompatibilité du build ou variation de bundle ; ne pas corriger automatiquement pendant cet audit.

4. Tests HTTP non destructifs

METHOD	URL locale	AUTH	Résultat / conclusion
GET	/api/healthz	Non requise	200 ; OPERATIONAL, database=true ; informations de santé limitées mais exposées
HEAD	/api/healthz	Non requise	200 ; mêmes en-têtes de sécurité et no-store
OPTIONS	/api/healthz	Non requise	200 ; Allow GET,HEAD ; wildcard CORS observé
GET	/api/installation	Non requise	200 ; profil public ; aucun secret observé localement
HEAD	/api/installation	Non requise	200 ; endpoint public confirmé
OPTIONS	/api/auth/login	Non requise	200 ; Allow POST ; aucune mutation
GET	/api/auth/session	Non requise	200 ; user=null ; session anonyme correctement gérée
GET	/api/diagnostics/health	Bearer requis	401 ; Token de diagnostic absent
HEAD	/api/e-commerce/bootstrap	Session requise	401 ; absence de session correctement bloquée
POST	/api/payments/diamanopay/webhook	Signature requise	401 ; signature invalide rejetée avant traitement

Les tests ont été exécutés sur 127.0.0.1:8080, sans cookie de session, sans jeton secret et sans payload susceptible de modifier une donnée. Le test webhook a volontairement utilisé un payload non signé ; il n'a pas atteint le traitement métier.

5. Points déjà correctement protégés

- SEC-001 : le mot de passe est haché côté serveur et les sessions de l'administrateur sont invalidées après mise à jour ; le défaut porte sur l'autorisation de l'appelant, pas sur le stockage du hash.
- Sessions : jeton aléatoire stocké sous forme de hash SHA-256, expiration serveur de 8 heures, cookie MAXIMUS HTTP-only, SameSite=Lax et Secure en production selon la configuration.
- Authentification : les routes privées refusent une session absente ; les comptes inactifs et entreprises inactives sont rejetés.
- Multi-entreprise : ResolveCompanyContext ne laisse pas un acteur non-MAXIMUS élargir son companyId ; les requêtes e-commerce consultées bornent les ressources par company_id.
- Installation : les tokens d'installation sont comparés via token_hash ; les routes d'installation centrale et de diagnostic ont des middleware dédiés.
- Diagnostics : GET /api/diagnostics/health refuse l'absence de Bearer token ; les erreurs de token ne divulguent pas la valeur attendue.

- Webhooks DiamanoPay : vérification HMAC avec hash_equals ; le traitement est idempotent pour les états déjà PAID/REFUNDED et les paiements utilisent des clés d'idempotence dans le service.
- Uploads : validations image/file avec extensions MIME autorisées et limites de taille observées pour les images ; les uploads dangereux de production n'ont pas été tentés.
- En-têtes : CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy et HSTS en production sont ajoutés par middleware ; les tests locaux les ont observés.
- Render : APP_DEBUG=false, DB_CONNECTION=pgsql, secrets non imprimés et stockage numérique prévu sur /var/data dans la configuration déclarative inspectée.

6. Tests non exécutés

- Production Render et PostgreSQL : TEST NON EXÉCUTÉ — absence d'autorisation de lecture opérationnelle directe dans cette phase et risque d'interagir avec des données réelles.
- Changement réel de mot de passe SEC-001 : TEST NON EXÉCUTÉ — l'action modifierait un compte sensible, même localement sans base de test dédiée.
- Création/confirmation de paiement, retrait, rejouement de webhook signé : TEST NON EXÉCUTÉ — risque financier et effet de bord externe.
- Upload multipart de fichier, chemin traversal et exécution serveur : TEST NON EXÉCUTÉ en environnement de production ; l'audit s'est limité au code et aux validations.
- IDOR/BOLA actif entre deux entreprises, permission matrix complète et parité Laravel/PostgreSQL : TEST NON EXÉCUTÉ de bout en bout ; les protections ont été inspectées et des tests existants ont été recensés.
- SAST : TEST NON EXÉCUTÉ au sens probant — le scanner a retourné une analyse incomplète avec zéro résultat ; ce zéro ne doit pas être interprété comme couverture complète.

7. Corrections proposées — minimales

- **Priorité 1** : corriger SEC-001 par un contrôle de rôle/policy côté serveur et une réauthentification pour le compte administrateur ; ajouter des tests négatifs sans mutation réelle.
- **Priorité 2** : requalifier le contrat public de /api/installation et /api/healthz ; conserver uniquement les champs nécessaires au bootstrap et au health check.
- **Priorité 3** : remplacer le CORS wildcard par une liste d'origines explicite, après inventaire des domaines central, entreprise et boutique.
- **Priorité 4** : traiter les dépendances signalées, en commençant par js-yaml 4.3.1, sans mise à jour automatique ; valider les chemins transitifs et le runtime réellement déployé.
- **Après correction** : relancer les tests Laravel avec PostgreSQL dédié, la matrice de rôles, les tests d'isolation, le typecheck, le build, l'audit de dépendances, le SAST complet et HoundDog.

8. Tests recommandés après correction

- Test HTTP authentifié avec employee, sector_manager, company_admin et maximus_admin sur PATCH /api/auth/company-password ; seul le périmètre autorisé doit obtenir 200.
- Test de réauthentification/expiration et invalidation des sessions de l'administrateur après changement.
- Test des réponses /api/installation et /api/healthz en central, dedicated et on_premise, avec contrôle d'absence de secrets et d'erreurs internes.
- Test CORS depuis chaque origine autorisée et depuis une origine inconnue, avec et sans credentials.
- Test d'isolation cross-company sur produits, commandes, fichiers, boutiques, transport et présences avec IDs d'une autre entreprise.
- Test webhook signé/non signé, double livraison, statut contradictoire et association charge/commande sur PostgreSQL de test.

9. Conclusion

AUDIT TERMINÉ — AUCUNE MODIFICATION EFFECTUÉE.

Le défaut SEC-001 est le résultat le plus important : la route de changement du mot de passe entreprise doit être corrigée avant toute nouvelle exposition à des utilisateurs non administrateurs. Les autres points sont des risques potentiels ou des alertes de dépendances à qualifier sans déduction abusive. La production Render/PostgreSQL n'a pas été altérée et n'est pas déclarée conforme sur la seule base de cet audit local.

Ce rapport ne contient aucune valeur de secret, aucun mot de passe, aucun token, aucune donnée financière ni donnée métier.